Cooking Buddy

Database Design

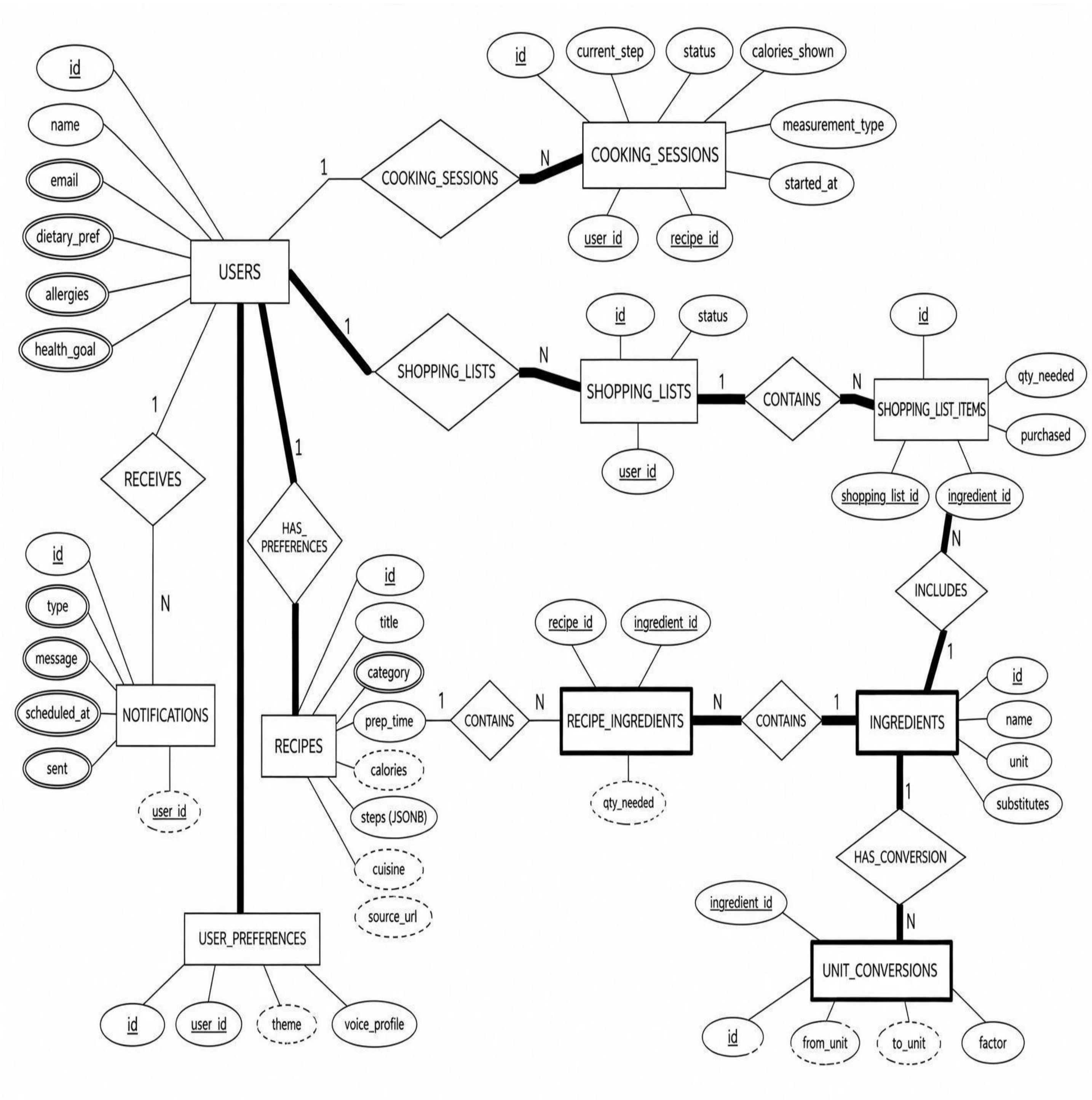
Why PostgreSQL Instead of MySQL

Point	PostgreSQL (chosen)	MySQL
Storing recipe steps	Can store steps as JSON and read one step at a time easily	Can store JSON too, but it is harder to search inside it
Following rules	Very strict about rules, so wrong or messy data is blocked	More relaxed, so bad data can slip in more easily
Works with Supabase	Built to work directly with Supabase, the hosting we use	Not made for Supabase, would need extra setup
Handling many linked tables	Good at handling many connected tables at once, like ours	Fine for simple apps, but gets harder with many linked tables
Advanced search	Good built-in search and filtering tools	Basic search tools, fewer advanced options
Growing the app later	Easy to add new features later without big changes	Bigger apps sometimes need extra tools added later
Cost	Completely free to use	Free version exists, but paid version is owned by Oracle
Community support	Large, active community and lots of documentation	Also large community, well known and widely used
Best fit for this project	Matches how Cooking Buddy stores and connects its data	Would work, but needs more extra effort to fit this project

What the Database Tracks

Table	What it's for
users	Who's using the web app, and their dietary needs
recipes	Recipes the web can suggest, with steps, calories, and cuisine tagging
ingredients	Every ingredient used across all recipes
recipe_ingredients	Which ingredients each recipe needs, and how much
cooking_sessions	A user's live progress while cooking a recipe, including which measurement tool they're using
shopping_lists	A shopping list a user has created
shopping_list_items	The individual items inside a shopping list
unit_conversions	Cup/scale conversion factors per ingredient, so quantities can be shown correctly either way
notifications	Scheduled reminders (meal planning, shopping list, cook time)
user_preferences	A user's saved theme and assistant voice profile

ER Diagram



id
name
email
dietary_pref
allergies
health_goal
USERS
1
COOKING_SESSIONS
N
id
current_step
status
calories_shown
measurement_type
COOKING_SESSIONS
started_at
user_id
recipe_id
1
SHOPPING_LISTS
N
id
status
SHOPPING_LISTS
user_id
1
CONTAINS
N
id
SHOPPING_LIST_ITEMS
qty_needed
purchased
shopping_list_id
ingredient_id
N
INCLUDES
1
1
RECEIVES
N
id
type
message
scheduled_at
sent
NOTIFICATIONS
user_id
1
HAS_
PREFERENCES
id
title
category
prep_time
RECIPES
calories
steps (JSONB)
cuisine
source_url
1
CONTAINS
N
recipe_id
ingredient_id
RECIPE_INGREDIENTS
qty_needed
N
CONTAINS
1
INGREDIENTS
id
name
unit
substitutes
1
HAS_CONVERSION
N
ingredient_id
UNIT_CONVERSIONS
id
from_unit
to_unit
factor
USER_PREFERENCES
id
user_id
theme
voice_profile

How the Tables Connect

Table	How it connects
users	The starting point — every session, list, notification, and preference set belongs to one user
cooking_sessions	Links one user to one recipe they're currently cooking, and records their measurement tool
recipes	Linked to ingredients through recipe_ingredients; tagged with cuisine for Sri Lankan sourcing
recipe_ingredients	Connects recipes and ingredients, with quantities
shopping_lists	One list per user
shopping_list_items	The items in a list, each tied to an ingredient
unit_conversions	Tied to ingredients; used whenever a session's measurement_type is 'cups'
notifications	Tied to a user; scheduled independently of any single session
user_preferences	One row per user; read by the frontend and by the Gemini voice API

foreign key relationships

Table	References
cooking_sessions	users, recipes
shopping_lists	users
shopping_list_items	shopping_lists, ingredients
recipe_ingredients	recipes, ingredients
unit_conversions	ingredients
notifications	users
user_preferences	users